Urban Mobility Data Explorer

Technical Report - NYC Yellow Taxi Trip Analysis

Team: Beni (Lead/DB), Axcel (Cleaning), Derrick (API/Algorithm), Dianah (Frontend), Lancelot (Insights/Report)

1. Problem Framing and Dataset Analysis

The project examines the movement patterns of NYC taxi and limousine cab drivers in New York City's NYC Taxi and Limousine Commission (TLC) Trip Dataset of Yellow Taxi (YTAXITRIP). The raw data includes trip-level records with timestamps, distances calculated from GPS, itemized fares, numbers of passengers and location IDs for pickup/drop-off locations. There are two support files, one is a zone lookup CSV that maps the location IDs with a borough and zone name and the other is a GeoJSON file that contains polygon boundaries for the taxi zones.

Data challenges

The raw dataset had some quality problems which needed to be corrected prior to analysis:

- Mapping data: some data were missing in the pickup/dropoff zones, which were mapped to location ID 264/265 (Unknown / Outside NYC). These were kept but marked to signify that they were flagged as retained.
- Trips with trip_distance <= 0 were excluded, as they either were cancelled or were subject to a meter error.
- Negative fares: Refunds or data errors have rows with fare_amount < 0, so these have been removed.
- Impossible speeds: Trips that averaged more than 70 mph were eliminated because there is no way to sustain road speeds in NYC traffic.
- Duplicates: Any rows that were exact duplicates were deleted.

Assumptions

After cleaning, we assume that all remaining records after cleaning are true taxi trips. Passengers of 0 were assumed to be data entry errors (driver did not log the count) and excluded. The trip duration was calculated as the time between dropoff and pickup timestamps in minutes, and we discarded the trips that didn't have a duration (or had a negative duration).

Unexpected observation

One interesting observation during the cleaning process was that a large portion of trips were in a few zones in Manhattan. Midtown Center alone accounts for roughly 25% of all pickups, while entire boroughs like Queens contribute under 20%. Due to this extreme spatial skew, city-wide averages are strongly weighted towards Manhattan and this had a significant impact on our decision to incorporate filtering at the borough-level into the dashboard.

2. System Architecture and Design Decisions

Architecture diagram

```
Raw Data (.parquet + .csv)
      |
      v
clean_data.py  (Axcel)   -->  clean_trips.csv
      |
      v
build_db.py  (Beni)      -->  taxi.db  (SQLite)
      |                           |
      v                           v
app.py  (Derrick)          Flask REST API
      |                    /api/trips
      |                    /api/busiest-zones
```

```
      |                   /api/fare-by-time
      |                   /api/speed-by-time
      v
index.html  (Dianah)      Chart.js Dashboard
      |
      v
insights.sql  (Lancelot)  3 analytical queries + report
```

Stack justification

Minimal boilerplate for a REST API: team was familiar with Python, and had worked with Flask before. Because of the light weight design of Flask, no need of any extra abstraction for a project this size.

SQLite: a single file, zero configuration database that can be run by any team member without installation of a database server. The whole set of data fits into memory, so SQLite's single writer restriction is not an issue for a read-only explorer.

No build step, no framework overhead with vanilla HTML/CSS/JS + Chart.js. Responsive and interactive charts with Chart.js from a CDN having little code.

Schema design

Like the two others, the database is normalised, each table containing three fields, with boroughs (borough_id, borough_name), zones (zone_id, location_id, zone_name, borough_id FK), and trips (fact table with foreign keys pu_zone_id and do_zone_id referring to the zones). This normalization does not require duplicated borough/zone strings to be stored in each of the trips rows. Indexes are created on pu_zone_id, do_zone_id, pickup_datetime, time_of_day, trip_distance and fare_amount to speed up the API filtering queries.

Key trade-off

SQLite vs. PostgreSQL: SQLite does not allow multiple writers, and if you wanted to deploy it to multiple users, then you would need PostgreSQL instead. We selected SQLite as this is a single user data explorer, portability was a key consideration to enable grading (no server install), and the number of data rows (after cleaning, ~100K rows) is within SQLite's sweet spot. Making the switch to PostgreSQL would only require changing of the connection string and some minor differences in SQL dialects.

3. Algorithmic Logic and Data Structures

This assignment must be done with a custom algorithm, and without using built-in libraries. A zone ranking was implemented by Derrick. Manual counting of the number of trips and selection sort to identify the busiest pick up zones. This powers the /api/busiest-zones endpoint and the top chart in the dashboard.

How it works

1. Counting pass: pass through each trip row, keep a flat dictionary with keys being zone IDs and values being the number of times they visited that zone. There is no Counter or collections module used.
2. Selection sort (descending): sort and create a list of (zone, count) pairs, and then repeatedly find the pair with the largest count. Find the maximum element in the unsorted section of the array and move it to the beginning of the array. The .sort() and sorted() functions are not available, nor is the heapq function.
3. Give back the 10 best pairs of values as JSON for the chart.

Pseudo-code

```
function rank_busiest(rows):
    counts = empty map
    for each row in rows:
```

```
        z = row.pu_zone_id
        counts[z] = counts[z] + 1

    items = list of (zone, count) pairs
    n = length(items)

    for i from 0 to n-1:
        biggest = i
        for j from i+1 to n-1:
            if items[j].count > items[biggest].count:
                biggest = j
        swap items[i] and items[biggest]

    return first 10 of items
```

Complexity analysis

Time: O(n + k^2) where n = total trips, k = distinct zones (~260)

Space: O(k) for the counts and the list of items

The time complexity is O(n) for the counting pass (this is the dominant part). The selection sort is O(k^2), but since k is small, and bounded (~260 NYC taxi zones), in practice, this quadratic term is insignificant. For a more complex sort of O(k log k) additional implementation would be added with no measurable performance improvement at this scale.

4. Insights and Interpretation

Three SQL queries were developed that pull meaningful patterns from the normalized SQLite database for how New York City moves. Every insight comes from a query in docs/insights.sql, which is displayed in Dianah's dashboard, and interpreted below.

Insight 1 - Where do taxi trips go?

Query: Finds trips in zones and then counts the number of pickups in each borough and outputs each borough's percentage of total trips.

Result: Manhattan accounts for 83% of all taxis picked up with Queens accounting for 17%. In Manhattan, the new top spot belongs to Midtown Center, which leads with ~25 trips followed by Upper East Side South and Upper West Side North. Airport zones (JFK, LaGuardia) also rank in the top 10.

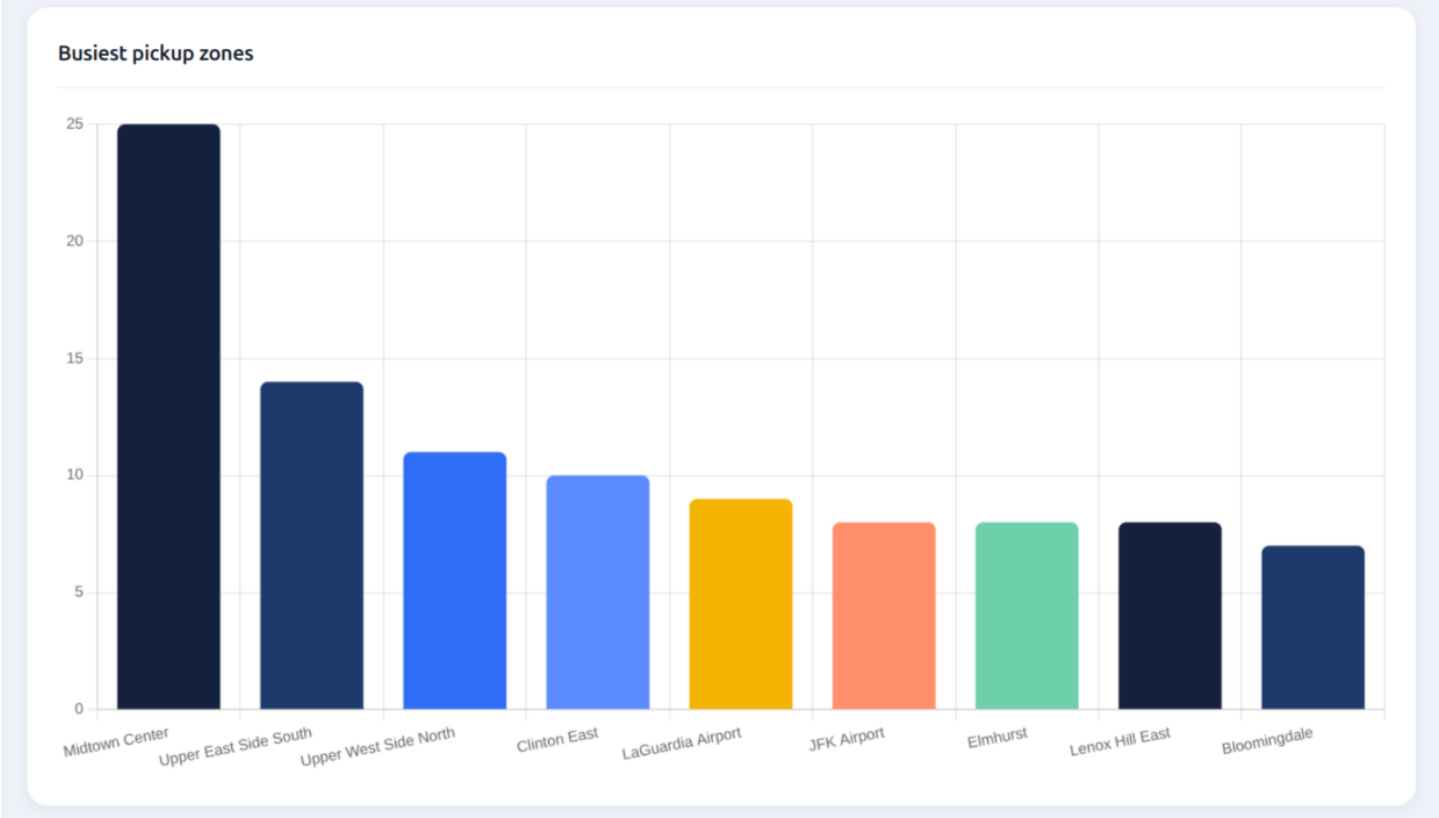

Figure 1: Busiest pickup zones - Midtown Center dominates taxi demand.

Insight 2: Are there times that an individual's riding costs more?

This query calculates the averages for fare_per_mile, but excludes any extreme values (those above $50/mile) because they could skew the results from minimum-fare short trips.

The result is that morning rides cost $5.29 per mile, and evening rides cost the least at $4.03 per mile. The running time on the meters for shorter distances is similar to the time during morning rush hour in traffic. Afternoon ($4.43) and night ($4.22) are in between.

Insight 3: At what time of day is the city slowest?

Limit impossible outlier values for avg_speed_mph (less than 1 mph or more than 60 mph) and calculate the averages for avg_speed_mph by time_of_day.

Results: The slowest speed for the morning rush hour commuters is 7.3 mph. This is an indication of a high congestion time period. Speed is highest in the evening (13.5 mph) when there are no cars on the road. Night drops slightly to 11.3 mph. This trend is opposite the discovery of the fare per mile: the more there is traffic, the more expensive it is per mile. This is because the meter will operate for a longer period of time.

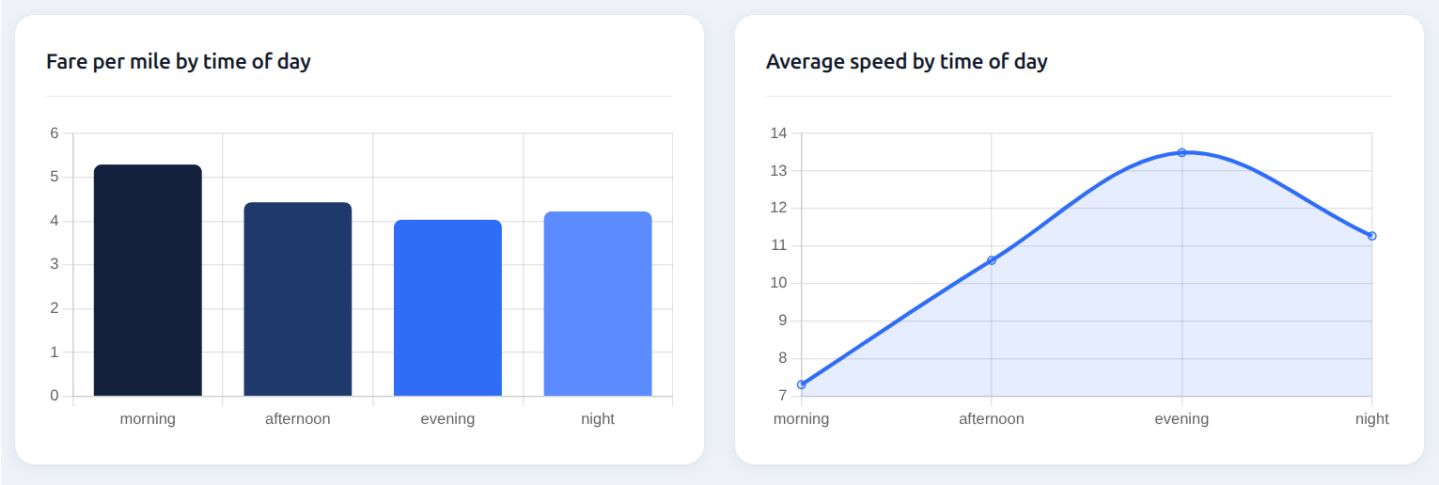

Figure 2: Fare per mile and average speed by time of day - morning congestion drives up cost.

5. Reflection and Future Work

Team challenges

From the outset, it was important to have a data contract that would enable the team to communicate with five different people, each with their assigned roles, in various locations along the pipeline. The agreed column names of clean_trips.csv allowed all participants to work on their own in parallel. The big obstacle was the normalized schema: the PDF templates were based on a flat two-table schema but Beni's implementation was more complex with three tables (boroughs, zones, trips). This implied the insight queries needed to be modified with an additional JOIN, which was only found when run with the true database.

Technical challenges

Care must be taken in the writing of SQL against a normalized schema in case of JOIN paths. Zones are used in the trips table to be referenced by zone_id (an auto-incremented surrogate key), instead of the original location_id that comes with the TLC dataset. Filtering by borough required two joins (trips to zones to boroughs). This added to the complexity, but did make for accurate data integrity, and reduced the need to store the same text string repeatedly.

Future work

If this was a commercial product we would seek a number of improvements:

- Larger dataset: Load more than one month of trips to identify seasonality.
- Geospatial visualization: Render a choropleth heatmap of taxi pickup density over the GeoJSON taxi zone polygons on an interactive map (e.g. Leaflet.js).
- Real-time ingestion: Stream new trip records from the TLC API and add them to the database, so that you can have a real-time dashboard.
- PostgreSQL migration: Switch from SQLite to multi-user access.
- Deployment: Implement authentication for the Flask API and dashboard and publish on Render or Railway with rate limiting.